

Create Interfaces for Contracts and Capabilities (Interview Notes)

What is an Interface?

An **interface** is a blueprint that defines a **contract** for a class. It specifies **what a class must do**, but not **how it does it**.

A class implements an interface using the implements keyword.

```
interface Flyable {  
    void fly();  
}  
  
class Bird implements Flyable {  
  
    @Override  
    public void fly() {  
        System.out.println("Bird is flying");  
    }  
}
```

Why Use an Interface?

Use an interface when:

- Different, unrelated classes should provide the same capability.
- You want to define a common contract for multiple implementations.
- You need **multiple inheritance** of behavior.
- You want to make your code loosely coupled and easier to maintain.

Real-world Example

```
interface Payment {  
  
    void pay(double amount);  
}
```

```

class CreditCardPayment implements Payment {

    @Override
    public void pay(double amount) {
        System.out.println("Paid using Credit Card");
    }
}

class UPIPayment implements Payment {

    @Override
    public void pay(double amount) {
        System.out.println("Paid using UPI");
    }
}

```

Both payment methods follow the same contract (pay()), but each provides its own implementation.

Important Interview Points

- An **interface defines a contract**, not an implementation.
 - A class uses the **implements** keyword to implement an interface.
 - A class can implement **multiple interfaces**, enabling multiple inheritance of behavior.
 - Interface methods are **public abstract by default** (unless declared default, static, or private in newer Java versions).
 - Interface variables are **public static final** (constants) by default.
 - An interface **cannot be instantiated** (no objects can be created directly).
 - Interfaces **do not have constructors** because they don't maintain object state.
 - Multiple unrelated classes can implement the same interface, promoting **polymorphism**.
 - Programming to an **interface rather than a concrete class** improves flexibility, maintainability, and testability.
-

Interface vs Abstract Class

Interface	Abstract Class
-----------	----------------

Defines a contract	Provides partial implementation
Uses implements	Uses extends
No constructors	Can have constructors
Supports multiple inheritance	Supports single inheritance
Best for capabilities	Best for shared code

Tricky Interview Questions

Q1. Why use an interface instead of an abstract class?

Answer:

Use an interface when you want to define a **capability or contract** that can be shared by unrelated classes. Use an abstract class when classes share common state and implementation.

Q2. Can we create an object of an interface?

Answer: No.

```
Payment p = new Payment(); // ❌ Compile Error
```

Q3. Can an interface have constructors?

Answer: No.

Interfaces don't create objects, so constructors are not allowed.

Q4. Can an interface have variables?

Answer: Yes, but every variable is automatically:

```
public static final
```

Example:

```
interface Test {  
    int MAX = 100;  
}
```

Equivalent to:

```
public static final int MAX = 100;
```

Q5. Can one class implement multiple interfaces?

Answer: Yes.

```
interface A {}  
interface B {}  
  
class Test implements A, B {}
```

Q6. Can an interface extend another interface?

Answer: Yes.

```
interface A {}  
  
interface B extends A {}
```

An interface can even extend **multiple interfaces**.

Q7. Why does Java allow multiple interfaces but not multiple class inheritance?

Answer:

Interfaces avoid the **Diamond Problem** because they don't inherit object state like classes

do. If two interfaces provide the same default method, Java forces the implementing class to override it and resolve the conflict explicitly.

Q8. Can an interface contain implemented methods?

Answer: Yes.

Since Java 8, interfaces can have:

- default methods
- static methods

Since Java 9, they can also have:

- private helper methods
-

Q9. Can an interface extend a class?

Answer: No.

An interface can only extend another interface.

Q10. What is the relationship between interfaces and runtime polymorphism?

Answer:

Interfaces are commonly used for runtime polymorphism.

```
Payment p = new CreditCardPayment();  
p.pay(1000);
```

The JVM decides at runtime which implementation of `pay()` to execute.

Interview Summary

- **Interface = Contract or Capability (What to do, not How to do it).**
- **Cannot create objects** of an interface.
- **Implemented using implements** keyword.
- **Supports multiple inheritance**, unlike classes.
- **All fields are public static final** (constants).

- **Abstract methods are public abstract** by default.
- Best used to achieve **loose coupling, flexible design**, and **runtime polymorphism**.
- **Choose an interface for capabilities (CAN-DO relationship)** and an **abstract class for shared implementation (IS-A relationship)**.

Interview One-Liner:

"An interface defines a contract that implementing classes must follow, enabling loose coupling, multiple inheritance, and runtime polymorphism without sharing implementation."